



**Hochschule
Kaiserslautern**
University of
Applied Sciences

Angewandte
Ingenieurwissenschaften
Kaiserslautern

Course of studies:

Master of Mechanical engineering / Mechatronics

Project Title:

System level rapid development in mechatronics

Submitted by:

Konda Madana Gopala Reddy
Matriculation No.: 889670

Mentor:

Prof. Dr.-Ing. Gerd Bitsch

Contents

1	ABSTRACT	4
2	INTRODUCTION	5
3	Modelling and Setup of the Inverted Pendulum System.....	6
3.1	Overview of the system	6
3.2	Building the Model in Simscape Multibody.....	6
3.2.1	World Configuration	6
3.2.2	Brick Setup	8
3.2.3	Joint Configuration.....	10
3.3	Analyzing Simulation Results	15
3.3.1	Base position displayed in Simulink Scope for different pendulum parameters	15
3.3.2	Actuation Setup.....	17
3.4	Preparing the model for the control design section.....	18
4	Control design	20
4.1	Overview	20
4.2	Building and Optimizing the Controller	20
4.2.1	Manual Gain Tuning and Results.....	22
4.3	Introducing Target Velocity to the Controlled Inverted Pendulum.....	25
4.3.1	Controller Behavior with Hand-Tuned Gains and Target Velocity	26
4.3.2	Base and Pendulum Position under Target Velocity	27
4.4	Linearizing the Simscape Multibody Model.....	28
4.4.1	Calculating the Feedback Gains in MATLAB.....	29
4.4.2	Base Velocity Response Using Model Linearizer Gains.....	30
4.5	Optimizing the Control Parameters.....	30
4.6	Step Response Validation in Control System Tuner	31
4.6.1	System Response after Control System Tuning.....	31

List Of Figures

FIGURE 3-1WORLD ENVIRONMENT SETUP IN SIMSCAPE MULTIBODY	7
FIGURE 3-2SUBSYSTEM_WORLD	8
9	
FIGURE 3-3.PENDULUM_BRICK	9
FIGURE 3-4.MODELLING OF BASE_BRICK	9
FIGURE 3-5PLANAR_JOINT	10
FIGURE 3-6.REVOLUTE_JOINT	10
FIGURE 3-7.NEW_FRAME_PENDULUM_BRICK	11
FIGURE 3-8.MODEL_SETUP	12
FIGURE 3-9.MECHANICAL_EXPLORERS	13
FIGURE 3-10.PENDULUM AT 5-DEGREE TILT	13
FIGURE 3-11. SUBSYSTEM STRUCTURE	14
FIGURE 3-12.HIERARCHY IN MECHANICS EXPLORER	14
FIGURE 3-14. CASE-1_PENDULUM_LENGTH	15
FIGURE 3-13. CASE-1_SCOPE	15
FIGURE 3-16. CASE-2_PENDULUM_LENGTH	16
FIGURE 3-15. CASE SCOPE	16
FIGURE 3-17.ACTUATION_SETUP	17
FIGURE 3-18. CASE1_POSITIVE_0.1_FORCE	17
FIGURE 3-19.CASE-2_NEGATIVE_0.1_FORCE	18
FIGURE 3-20.SIMSCAPE_MODEL_WITH PRISMATIC JOINT	18
FIGURE 3-21. MECHANICS EXPLORER VIEW SHOWING UPRIGHT PENDULUM AND ALIGNED COORDINATE SYSTEMS AFTER PRISMATIC JOINT SETUP	19
FIGURE 4-1.FINAL_SIMSCAPE LAYOUT WITH FEEDBACK_CONTROLLER	20
FIGURE 4-2. BASIC FEEDBACK CONTROL STRUCTURE FOR THE INVERTED PENDULUM WITH FOUR STATE VARIABLES AND CORRESPONDING GAIN VALUES	21
FIGURE 4-3BASE VELOCITY RESPONSE SHOWING QUICK DAMPING AND STEADY-STATE STABILIZATION AFTER PENDULUM BALANCING	23
FIGURE 4-4 BASE POSITION RESPONSE SHOWING SMALL CORRECTIVE DISPLACEMENT AND SMOOTH RETURN TO EQUILIBRIUM UNDER HAND-TUNED CONTROL	23
FIGURE 4-5.PENDULUM_ANGULAR_VELOCITY	24
FIGURE 4-6PENDULUM_ANGULAR_POSITION	24
FIGURE 4-7IMPLEMENTATION OF TARGET VELOCITY USING A PULSE GENERATOR CONNECTED TO THE CONTROL LOOP	25
FIGURE 4-8COMPARISON BETWEEN BASE VELOCITY AND TARGET VELOCITY	26
FIGURE 4-9COMPARISON OF BASE AND PENDULUM POSITIONS SHOWING STABLE MOTION	27
FIGURE 4-13.MATRIX_A	28
FIGURE 4-12.MATRIX_B	28
FIGURE 4-10.MATRIX_D	28
FIGURE 4-11.MATRIX_C	28
FIGURE 4-14COMPARISON OF BASE VELOCITY (PURPLE) AND TARGET VELOCITY (GREEN) USING OPTIMIZED GAINS OBTAINED FROM MODEL LINEARIZER	30
FIGURE 4-15.STEP TRACKING RESPONSE BETWEEN TARGET VELOCITY AND BASE VELOCITY AFTER TUNING USING CONTROL SYSTEM TUNER	31
FIGURE 4-16.OMPARISON OF BASE VELOCITY AND TARGET VELOCITY AFTER CONTROL SYSTEM TUNING	31
FIGURE 4-17.OMPARISON OF BASE VELOCITY AND TARGET VELOCITY AFTER CONTROL SYSTEM TUNING	31

1 ABSTRACT

In this project, I worked on designing and simulating a self-balancing robot using MATLAB Simulink and Simscape Multibody. Since balancing robots behave like inverted pendulums in the real world, I used that concept to model and test the system virtually. The goal was to keep the pendulum upright while also allowing the base to move according to a target velocity.

After building the mechanical model in Simscape, I linearized it around the upright position to obtain the state-space representation. Using these matrices, I designed an LQR controller that calculates how much force should be applied to keep the pendulum balanced. Each state—base position, base velocity, pendulum angle, and angular velocity—was fed back into the controller to generate a stable response.

Through several tuning steps and simulations, the system successfully stabilized the pendulum and followed the desired velocity commands. The results show that this model-based LQR control approach is reliable and forms a strong foundation for real self-balancing robots, such as the two-wheel platform used in the System Level Rapid Development module.

2 INTRODUCTION

You know how hard it is to balance a broomstick on your fingertip? The moment you stop paying attention, it just falls. That's pretty much what my project is about getting a robot to balance itself like that. Think of those self-balancing scooters or Segways; they're basically doing the same thing, just with wheels and motors.

For my project, I built a virtual version of that balancing act in MATLAB Simulink using the Simscape Multibody toolbox. Instead of an actual robot, I used what's called an inverted pendulum basically a stick standing on a sliding base. The base can move left and right to stop the stick from tipping over. It's a simple idea but surprisingly tricky to get right.

The goal was to design a control system that not only keeps the pendulum upright but also lets the base move at a specific speed kind of like telling a balancing robot to drive forward smoothly without falling. To do that, I linearized the system around the upright position and used something called an LQR controller to calculate how much force to apply.

Honestly, it was pretty cool to see how something that looks unstable and chaotic can actually be controlled so precisely. It gave me a better appreciation for how balancing robots work in real life it's all about continuous corrections, feedback, and a lot of trial and error

3 Modelling and Setup of the Inverted Pendulum System

3.1 Overview of the system

So, the system I worked with is a simple but classic setup an inverted pendulum built in Simscape Multibody. It's basically my virtual version of a self-balancing robot. The model has two main parts: a base brick and a pendulum brick.

The base brick is like the robot's wheels or platform. It can slide left and right along the x-axis but can't move in any other direction. On top of it, I attached the pendulum brick, which represents the robot's body. These two parts are connected through a revolute joint, so the pendulum can freely swing back and forth as gravity tries to pull it down.

Now, here's the tricky part this setup is naturally unstable. The pendulum doesn't stay upright on its own for even a moment; it just falls if you don't control it. To keep it balanced, the base has to react instantly, moving just enough in the right direction whenever the pendulum starts to tilt. That's what makes this system so interesting to work with. It might look simply, but getting it to stay upright is a real control challenge — and honestly, that's what made building and tuning this model so satisfying.

3.2 Building the Model in Simscape Multibody

3.2.1 World Configuration

I first set up the world environment for the simulation. This part defines how everything interacts the coordinate system, gravity, and solver settings. Started by creating a new Simscape Multibody model using the MATLAB command:

```
smnew('my_inverted_pendulum')
```

Running this command automatically opens a blank template with the basic world elements already in place. It's a nice shortcut because it gives

you a ready-to-use framework instead of starting completely from scratch. The generated model includes three essential blocks:

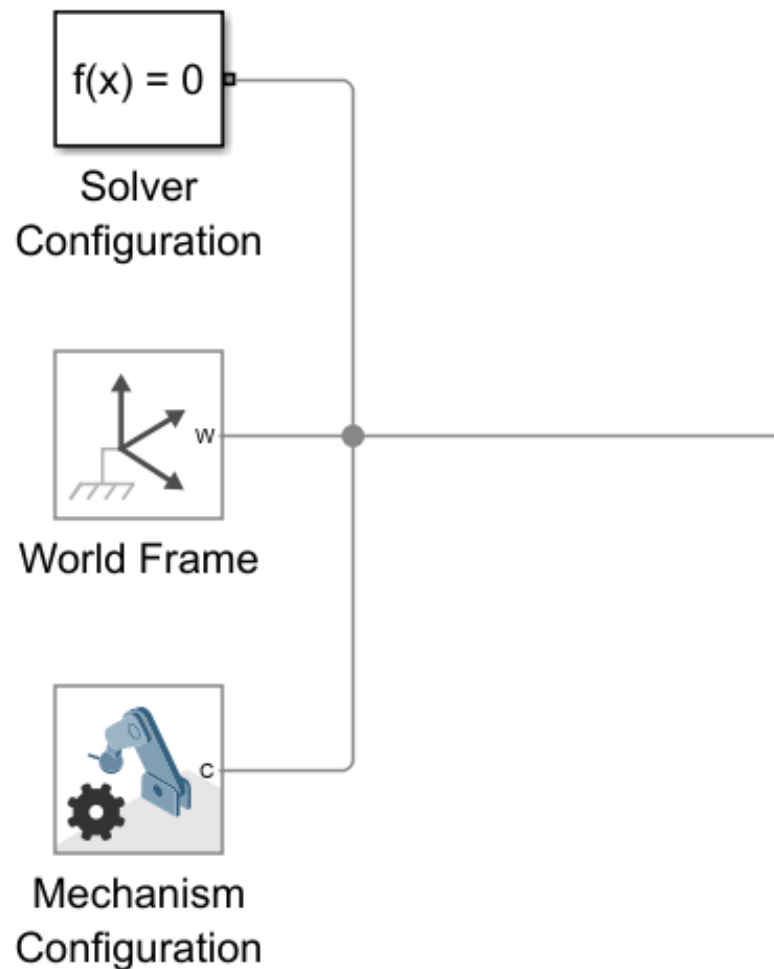


Figure 3-1 World Environment Setup in Simscape Multibody

1. **Solver Configuration,**
2. **World Frame**
3. **Mechanism Configuration.**

Solver Configuration block: handles all the numerical settings for the simulation. It basically tells MATLAB how to compute motion and time progression during the simulation.

World Frame: block serves as the global reference for all the mechanical parts that I'll add later. Everything joints, solids, and transforms connects back to this frame.

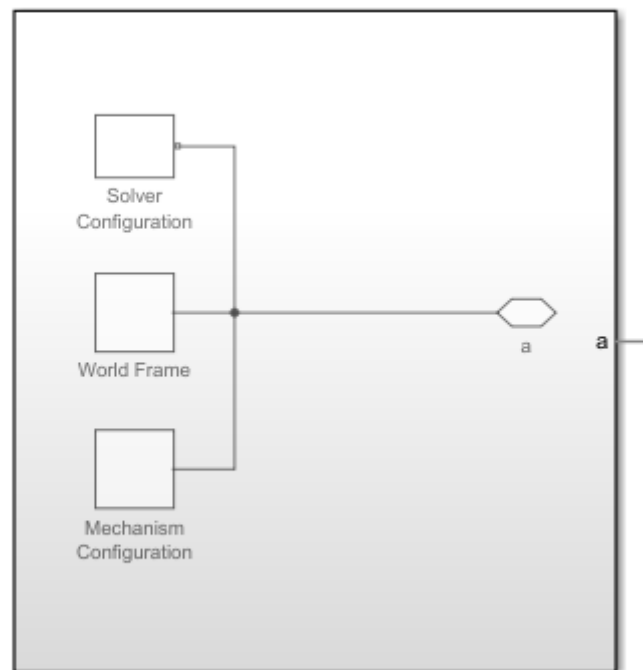


Figure 3-2subsystem_world

Mechanism Configuration: block defines the physical environment, including gravity. I kept gravity at -9.81 m/s^2 in the z-direction, which matches real-world conditions.

I placed these three blocks inside a subsystem called “**subsystem_world.**” Once everything was connected, I linked the world frame port ‘a’ from this subsystem to the rest of my model. This step ensures that every component in the simulation — from the base to the pendulum — uses the same world reference and gravity settings.

3.2.2 Brick Setup

I constructed the base brick after arranging the joints. The base is a representation of the horizontally moving cart or platform that holds the pendulum. It's the component that maintains the system's equilibrium and reacts to the control force.

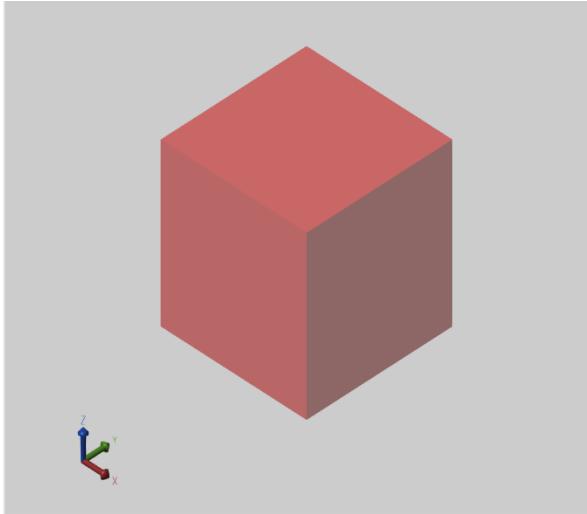


Figure 3-4.modelling of base_brick

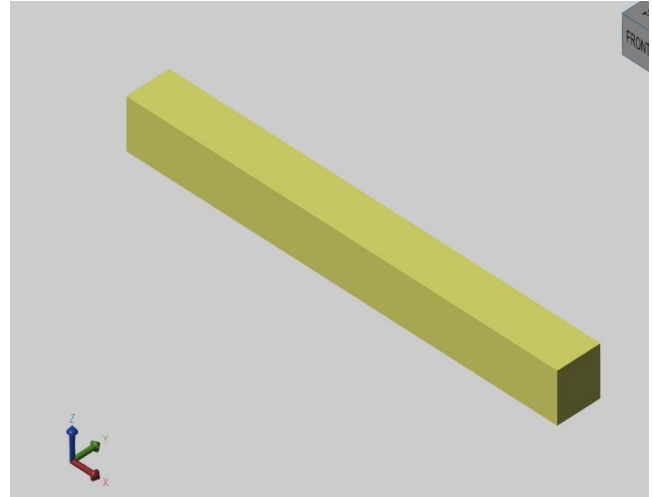


Figure 3-3.Pendulum_Brick

To build the main structure of my inverted pendulum model, I started with two simple blocks the base brick and the pendulum brick. These two are basically the heart of the setup, and everything else connects to them later.

I began with the base brick, which is the small red cube you can see in *Figure 3.4*. I made it using a solid block in Simscape Multibody with dimensions of $0.05\text{ m} \times 0.05\text{ m} \times 0.05\text{ m}$. It's small but works perfectly as the platform for the pendulum. After creating it, I added a coordinate system right at its centre of gravity (CG) using a Rigid Transform block. Then, I placed another coordinate system 2 cm above the CG, which will later help when attaching the pendulum. It's a simple trick, but it really helps keep everything lined up nicely.

Next, I worked on the pendulum brick, shown in *Figure 3.3*. This one's a bit taller and thinner $0.3\text{ m} \times 0.02\text{ m} \times 0.02\text{ m}$ and I made it yellow just to tell it apart from the base. I also created a local coordinate system near the end of the pendulum, about 1 cm before the tip, so I'd have a clean connection point when linking it to the base.

After setting both bricks up, I ran the simulation just to check if everything looked right. Nothing moved yet, of course, but I could see both solids and their coordinate systems clearly in the Mechanics Explorer

3.2.3 Joint Configuration

In Simscape Multibody, joints are what define how the different parts of a model can move and interact with each other. Each joint provides specific degrees of freedom, and choosing the right ones is what makes the system behave realistically.

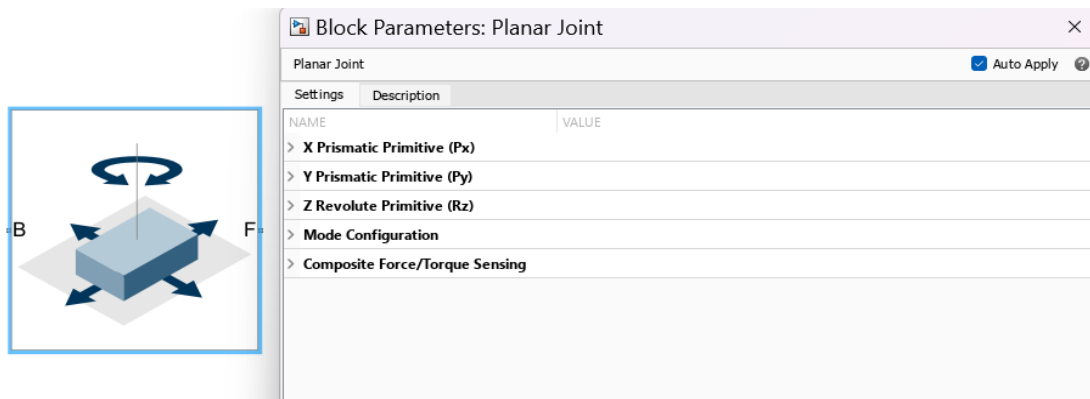


Figure 3-5: planar_joint

For the initial setup, I used a planar joint along with a revolute joint to get familiar with the system and test how the model behaves. The planar joint connects the base brick to the world frame and allows it to move freely in the horizontal plane. It lets the base translate along the x and y axes and rotate about the z-axis.

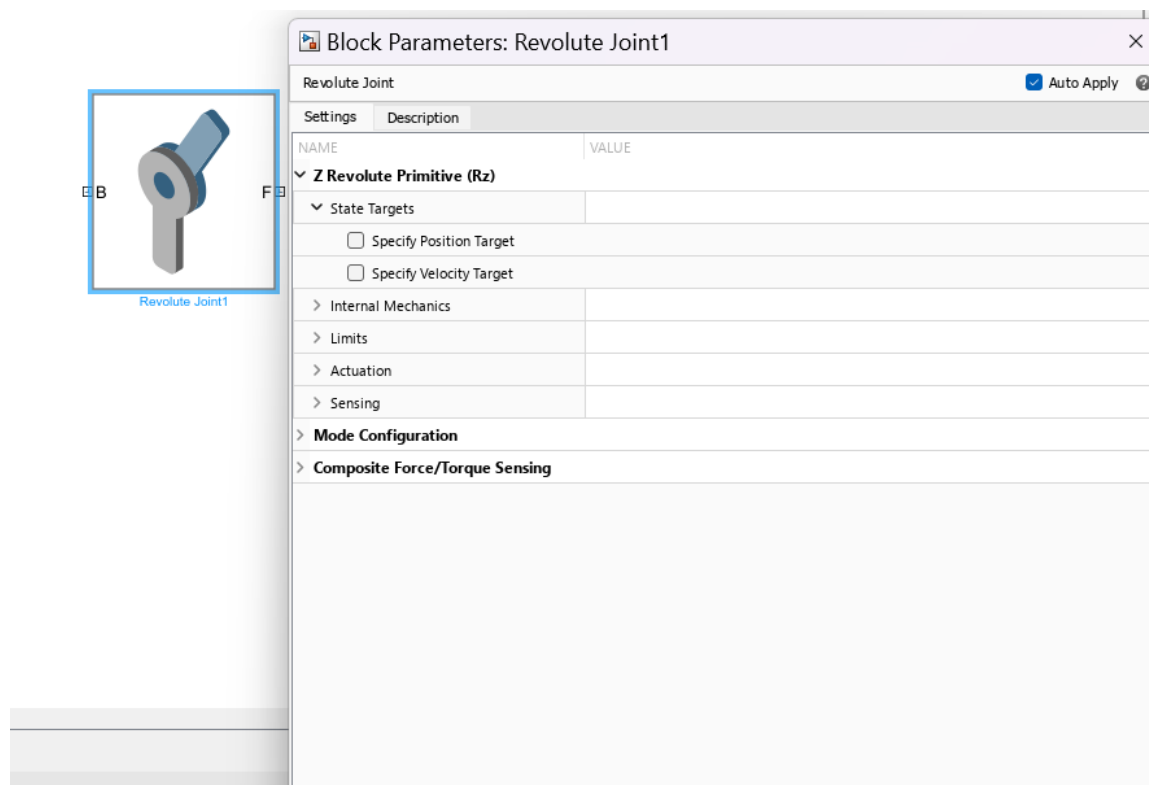


Figure 3-6: revolute_joint

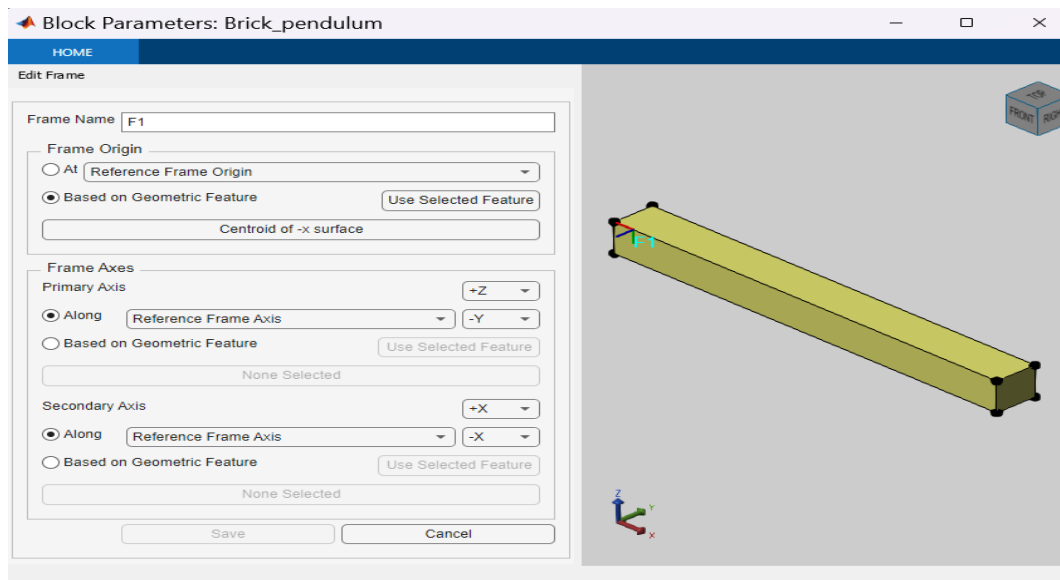


Figure 3-7.new_frame_pendulum_brick

I also added a revolute joint between the base brick and the pendulum brick. This joint gives the pendulum one rotational degree of freedom, allowing it to swing forward and backward in the x-z plane.

In Simscape Multibody, the revolute joint allows rotation only around the z-axis of its connected frames. This means that if the coordinate frames on the solids aren't aligned properly, the pendulum might not rotate in the desired plane.

To fix this, I adjusted the frame orientation on the pendulum brick by creating a new frame at the end of the pendulum, which serves as the connection point for the revolute joint. The new frame was placed at the centroid of the -x surface and oriented as shown in *Figure 3-7*.

In this configuration:

- The Primary Axis was set to +Z,
- The Secondary Axis was set to +X,
- Both were aligned with the corresponding reference frame directions.

After setting up the new frame, I used a Rigid Transform block to position the pendulum in its upright orientation. I rotated it by -90° around the z-axis, which ensured that the pendulum stood vertically when the simulation started. This setup made the model visually correct and ready for motion testing once the joints were added

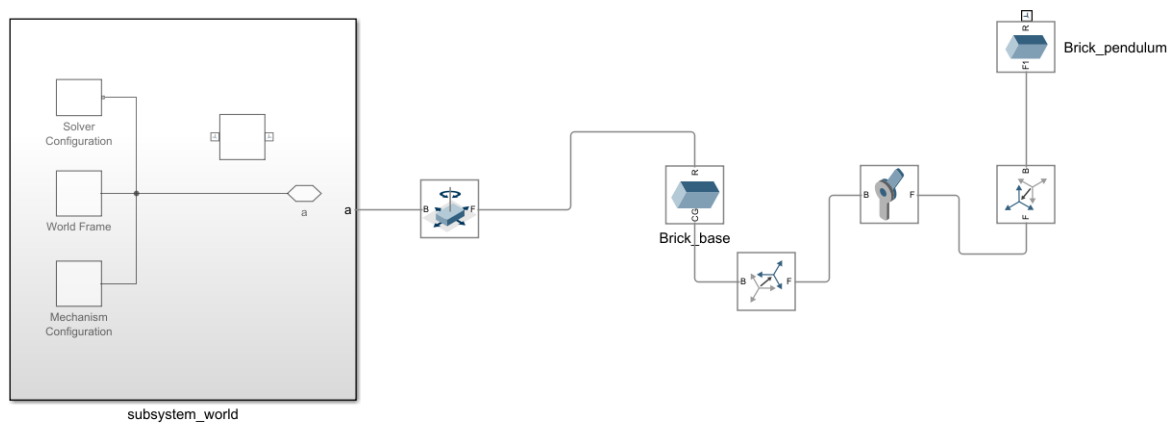


Figure 3-8.model_setup

After defining all the individual components the world configuration, joints, and the two solid bricks I assembled them into a complete system as shown in Figure 3-8.

The setup begins with the world subsystem, the output port 'a' of the world subsystem is connected to the rest of the system to serve as a global reference frame.

Next, a planar joint is connected directly to the world subsystem. This joint links the Brick base to the world and allows the base to move freely in the horizontal plane, and a Rigid Transform block is used to added a small offset of 2 cm above the base's centre of gravity (CG) to make room for attaching the pendulum later On top of the base, a revolute joint connects the Brick pendulum. This joint allows the pendulum to rotate freely in the x-z plane. To ensure the pendulum behaves correctly, an additional Rigid Transform block was used to align its frame orientation and keep it upright at the start of the simulation.

In this configuration, the system is complete:

- The base can move in the horizontal direction.
- The pendulum can swing about the revolute joint.

The world subsystem provides the fixed reference and gravitational field.

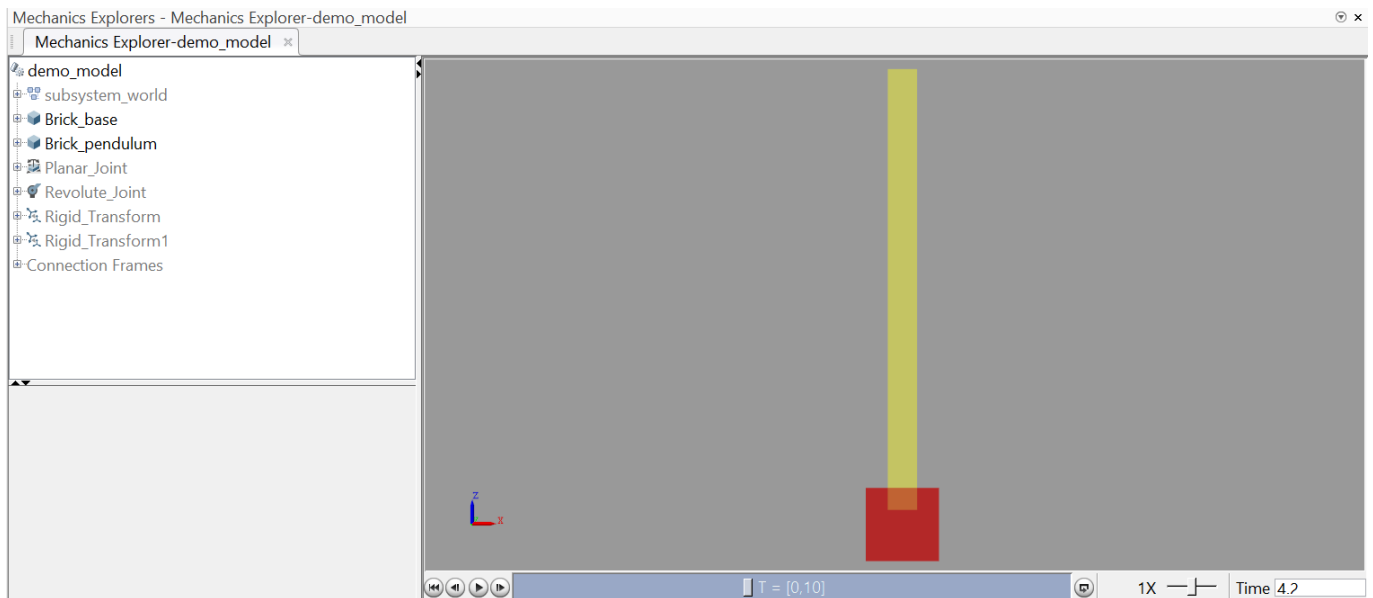


Figure 3-9.Mechanical_explorers

Running the model in the Mechanics Explorer confirmed that all the frames were properly connected and oriented. The pendulum stood upright and was free to swing naturally, while the base responded correctly to movement in the horizontal direction. This verified that the mechanical structure of the inverted pendulum system was correctly assembled and ready for control design

3.2.3.1 Adapting Model Parameters

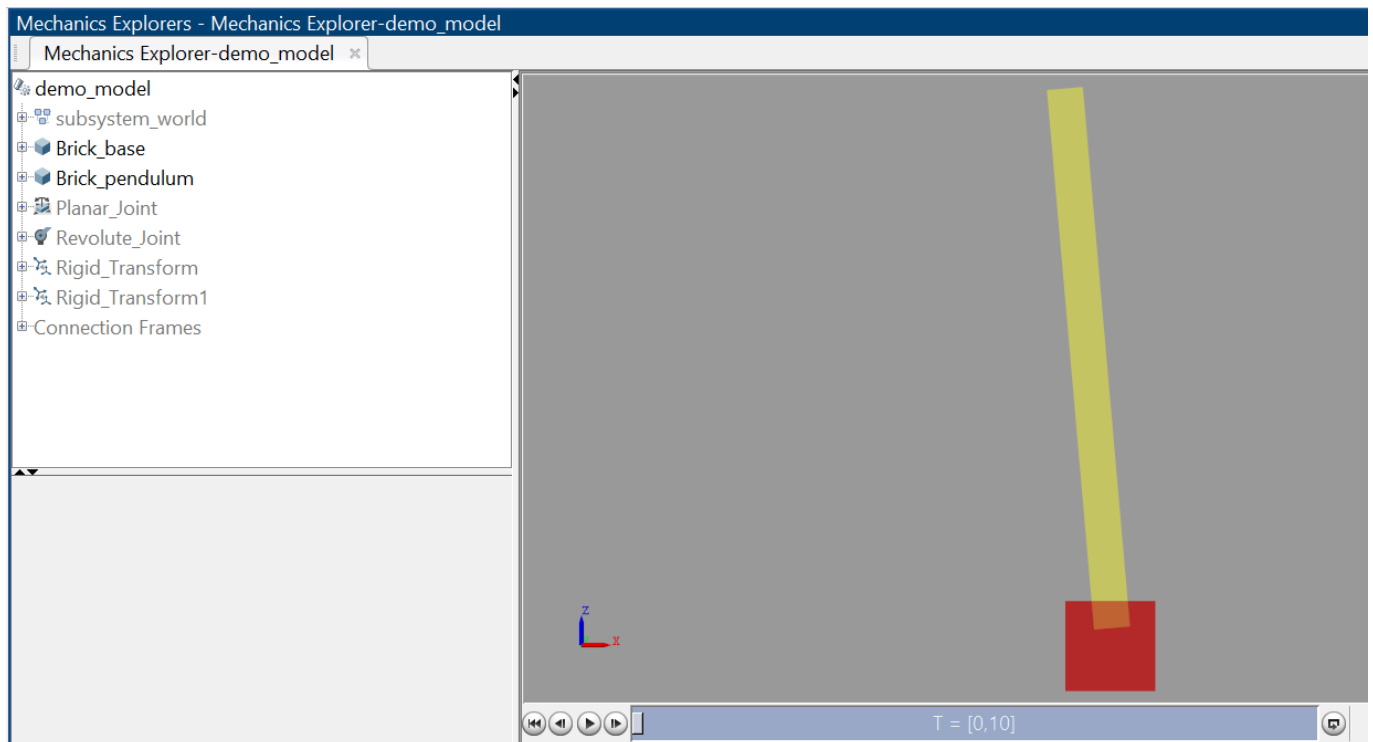


Figure 3-10.pendulum at 5-degree tilt

After completing the basic setup, I gave the pendulum a small 5° initial tilt using the revolute joint's state target. This slight offset helps the pendulum

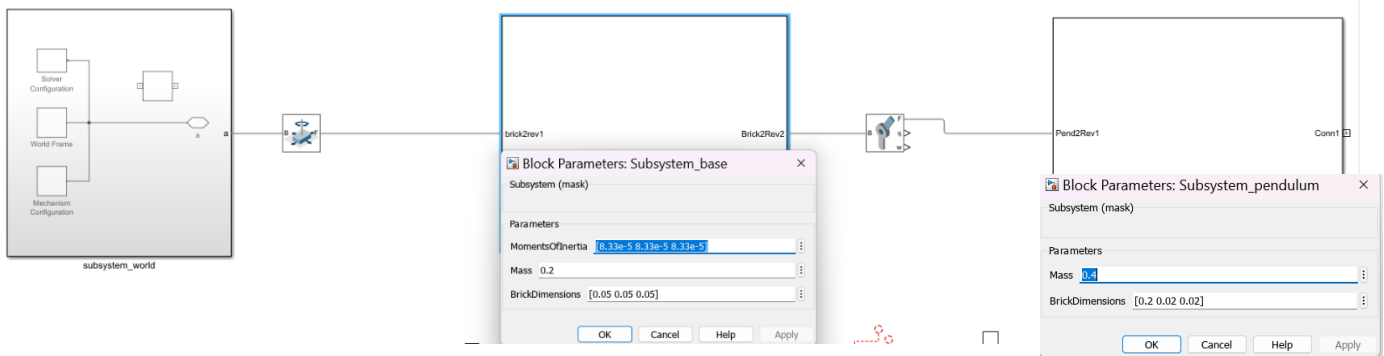


Figure 3-11. Subsystem structure

start in an almost upright position but just off balance enough to move when the simulation begins

Once that was set, I decided to organize the model into three subsystems — the base, the pendulum, and the world. Separating the model this way makes it much easier to manage and update later. I then masked each subsystem so that parameters like mass, dimensions, and inertia could be adjusted directly from the mask dialog without opening each block

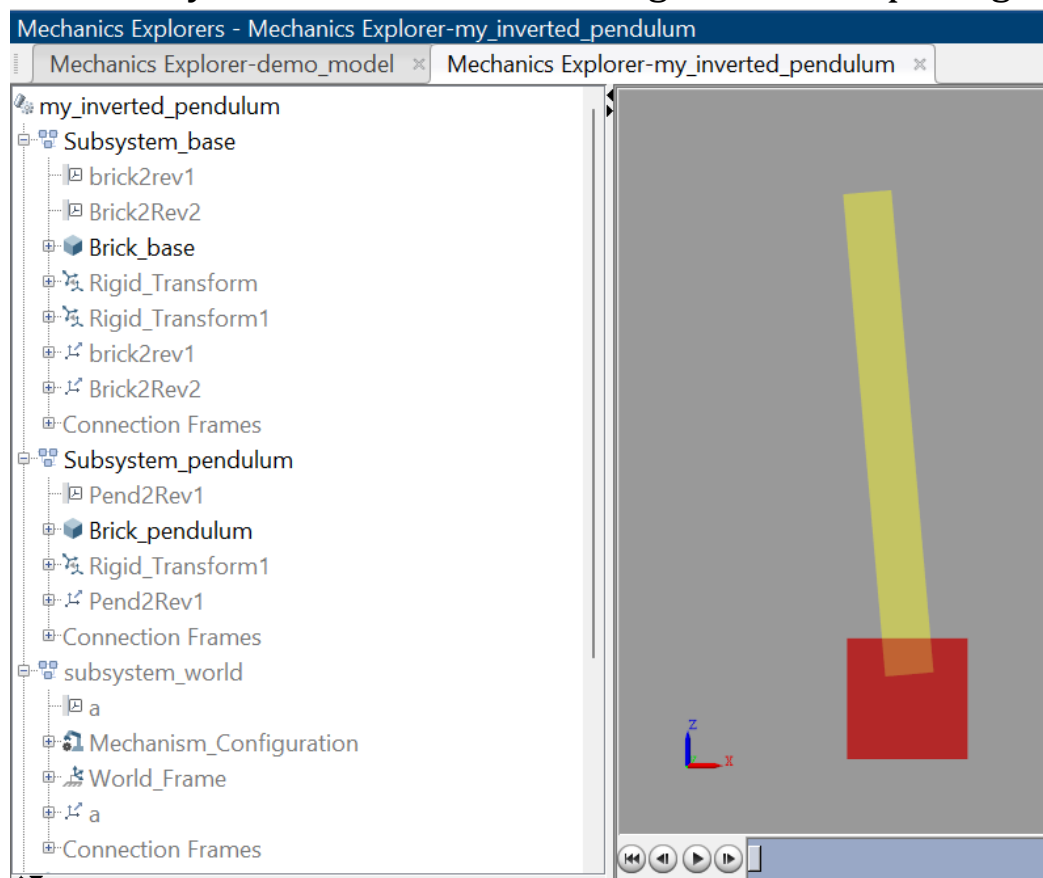


Figure 3-12. Hierarchy in Mechanics Explorer

manually. It really simplified the process of tuning the model and testing different configurations.

After saving these changes, I ran the simulation and checked the model in Mechanics Explorer. The structure tree, shown in *Figure 3.12*, confirmed that all the subsystems were properly connected.

3.3 Analyzing Simulation Results

After completing the setup and verifying that the system worked correctly, I started analyzing how the base reacted when the pendulum's physical parameters were changed. The main focus was to observe how the base movement varied when I adjusted the length and mass of the pendulum. To visualize the motion, I enabled the sensing options in the prismatic joint and used a PS-Simulink Converter to send the position data to a Simulink Scope. This allowed me to track the base's position over time and clearly see how it moved in response to the pendulum's oscillations.

3.3.1 Base position displayed in Simulink Scope for different pendulum parameters

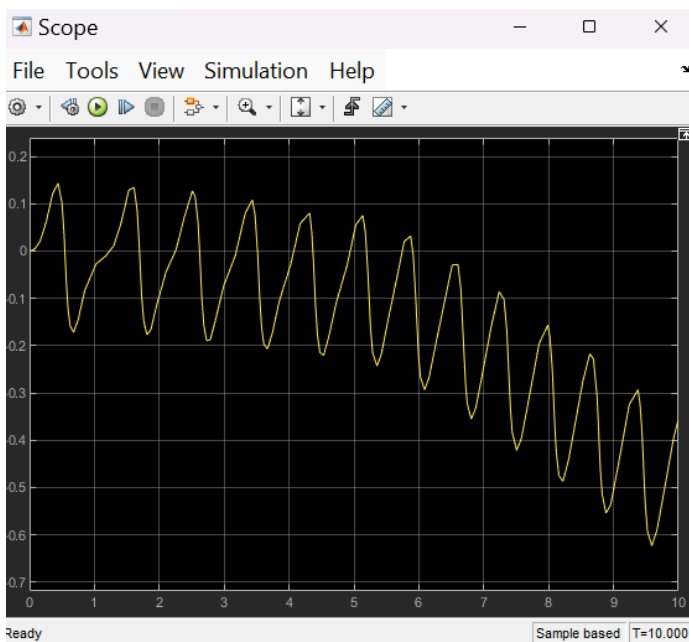


Figure 3-13. CASE-1_Scope

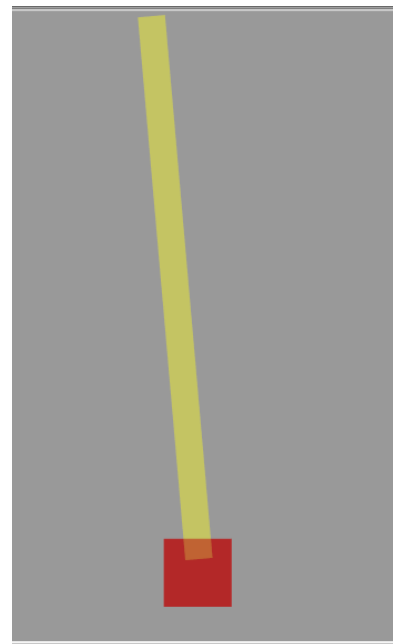


Figure 3-14. CASE-1_pendulum_length

3.3.1.1 Case 1: Pendulum Length = 0.4 m, Mass = 1 kg

For this configuration, I set the pendulum length to 0.4 meters and the mass to 1 kilogram to observe how the base reacts to a moderately long and heavy pendulum. The scope in Figure 3-13 shows the base displacement over time. From the graph, it's clear that the base initially moves with a

noticeable oscillation as it reacts to the pendulum's swinging motion. The amplitude of the base movement gradually decreases as time progresses

3.3.1.2 case: $L = 0.2$ m and $M = 0.4$ kg.

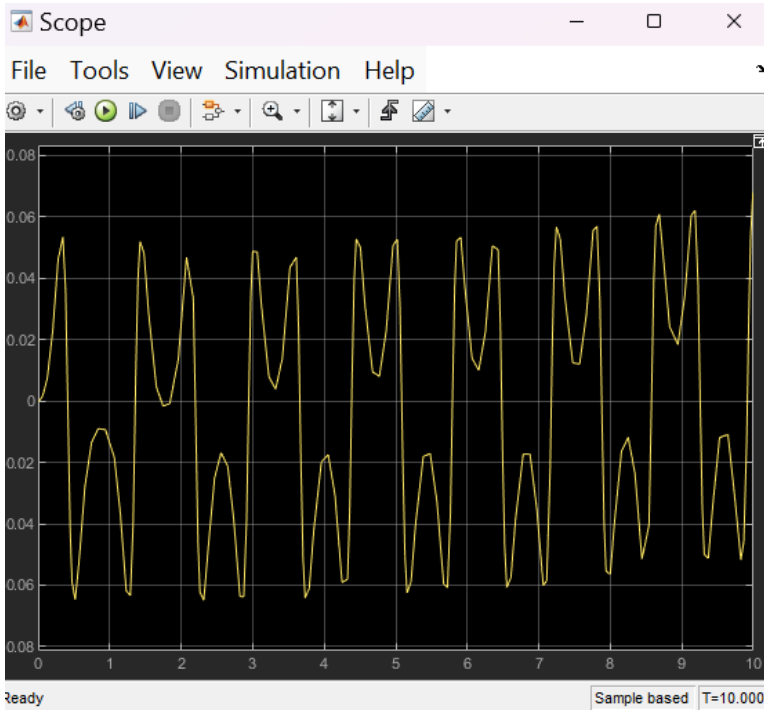


Figure 3-15. Case scope

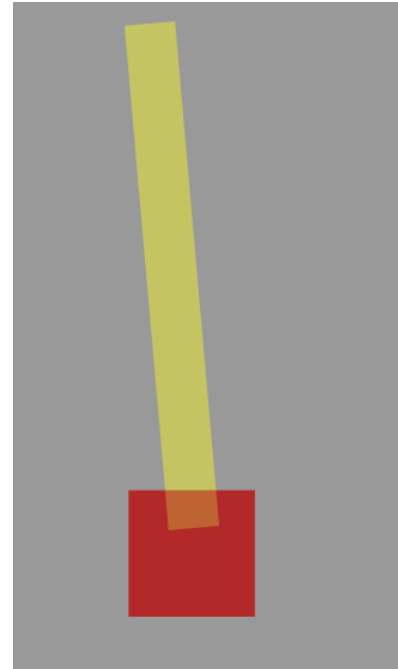


Figure 3-16. CASE-2_Pendulum_Length

In this simulation, I reduced the pendulum length to 0.2 m and its mass to 0.4 kg to see how a lighter and shorter pendulum would affect the base motion. The resulting base displacement is shown in Figure 3-15.

Compared to the previous case, the oscillations here are much faster and tighter, with a noticeably higher frequency. This happens because a shorter pendulum has a smaller moment of inertia, allowing it to swing back and forth more quickly.

3.3.2 Actuation Setup

Once the system's free motion was verified, the next step was to see how it behaves when an external force is applied. In Simscape, setting up actuation works almost the same way as sensing — you just enable the actuation option in the joint settings.

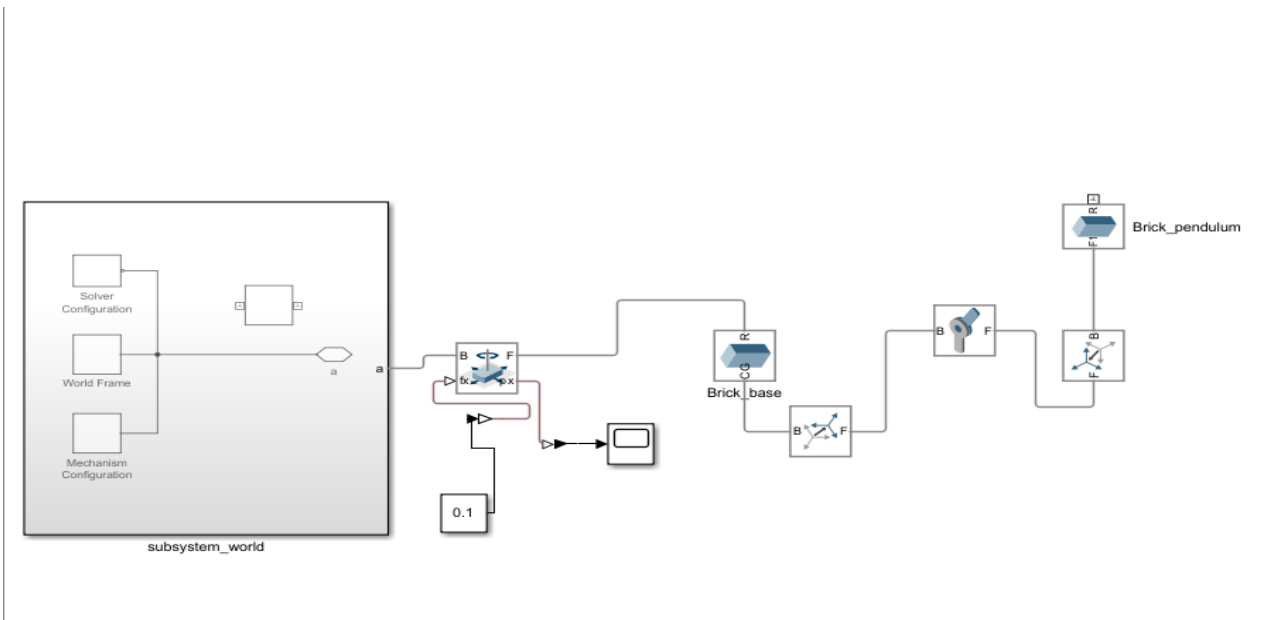


Figure 3-17. Actuation_setup

After verifying the model's free motion, I tested how the system reacts when an external force is applied to the base. In Simscape, actuation works similarly to sensing you enable it in the joint block

To generate the input, I used a Constant block from Simulink set to a value of 0.1 and connected it through a Simulink-PS Converter. This setup converts the Simulink signal into a physical signal that the Simscape environment can interpret.

3.3.2.1 Case 1: Actuation Force = +0.1 N

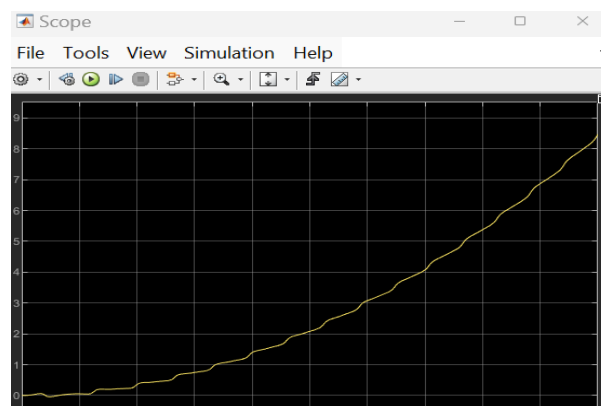


Figure 3-18. case1_positive_0.1_force

The scope output, shown in Figure 3-18, displays a smooth and continuously increasing displacement curve. This steady rise indicates that the base keeps accelerating as the constant force is applied—there's no counteracting control yet, so the base doesn't settle at a fixed position. The response confirms that the actuation setup is functioning correctly and that the applied force is being transmitted through the converter into the physical model.

This test also highlights the natural open-loop behaviour of the system: even a small, constant force is enough to keep the base moving continuously, showing just how sensitive and unstable the inverted pendulum is without feedback control

3.3.2.2 Case 2: Actuation Force = -0.1 N

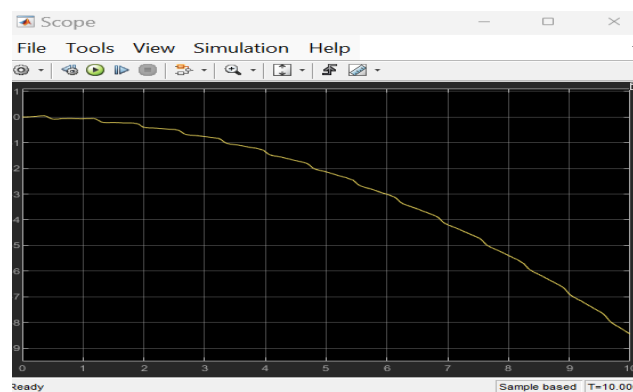


Figure 3-19. CASE-2_NEGATIVE_0.1_FORCE

As expected, the base began to move smoothly in the opposite direction, as shown in Figure 3.x. The displacement curve now decreases steadily over time, mirroring the result from the positive force case. This confirms that the system behaves symmetrically when the actuation direction is reversed.

3.4 Preparing the model for the control design section

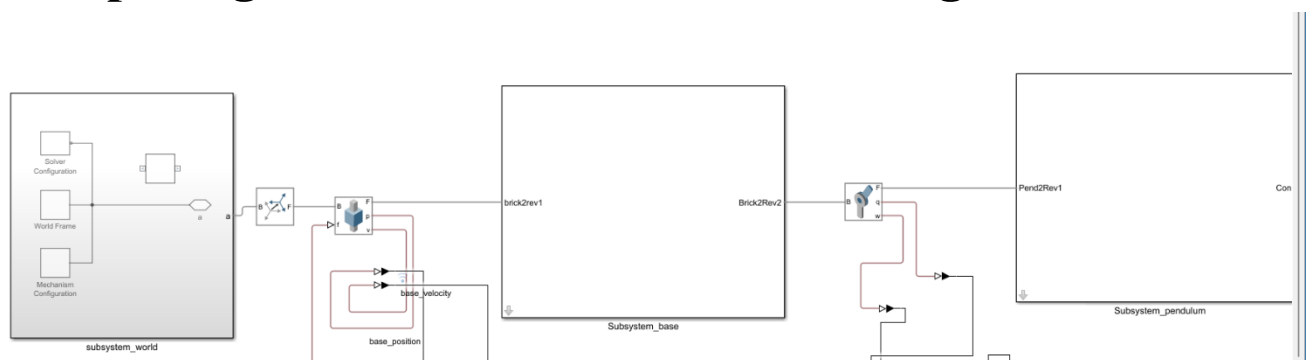


Figure 3-20. simscape_model_with prismatic joint

The earlier setup with the planar joint allowed free movement in both x and y directions and rotation around the z-axis, which was great for

understanding the system's basic behaviour. However, for control implementation, we need a more realistic configuration where the base moves only along the x-direction — just like in a real inverted pendulum system.

To achieve this, I replaced the planar joint with a prismatic joint, which allows translation along a single axis. By default, Simscape defines this motion along the z-axis, so I inserted a Rigid Transform block between the prismatic joint and the world frame to reorient the movement direction. This transform effectively replaced the z-axis motion with x-axis motion, ensuring that the base now slides horizontally instead of vertically.

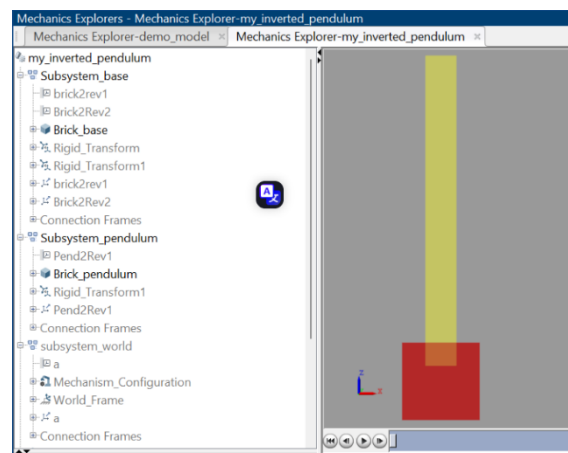


Figure 3-21. Mechanics Explorer view showing upright pendulum and aligned coordinate systems after prismatic joint setup

Once the prismatic joint and rigid transform were correctly configured, I checked the coordinate system alignment. I set up all the coordinate systems (CS) so that the upright position of the pendulum corresponds to zero degrees can clearly show in Figure 3-21

4 Control design

4.1 Overview

After verifying the mechanical model and ensuring all components worked as expected, the next step was to design a control system that could keep the pendulum balanced in its upright position.

To achieve this, the nonlinear Simscape model was linearized around the upright equilibrium point. This linear approximation allowed the system to be expressed in state-space form, which provides a convenient mathematical framework for controller design. Using this representation, a state-feedback controller was developed to stabilize the pendulum by continuously adjusting the base motion based on its current position, velocity, and the pendulum's angular states.

4.2 Building and Optimizing the Controller

Once the model was linearized, the next step was to build the actual controller that keeps the pendulum balanced. The basic control concept is straightforward: each of the four state variables base position, base velocity, pendulum angle, and pendulum angular velocity is multiplied by its own specific gain value. The resulting signals are then summed up, and the negative of this sum is applied as a corrective force to the base.

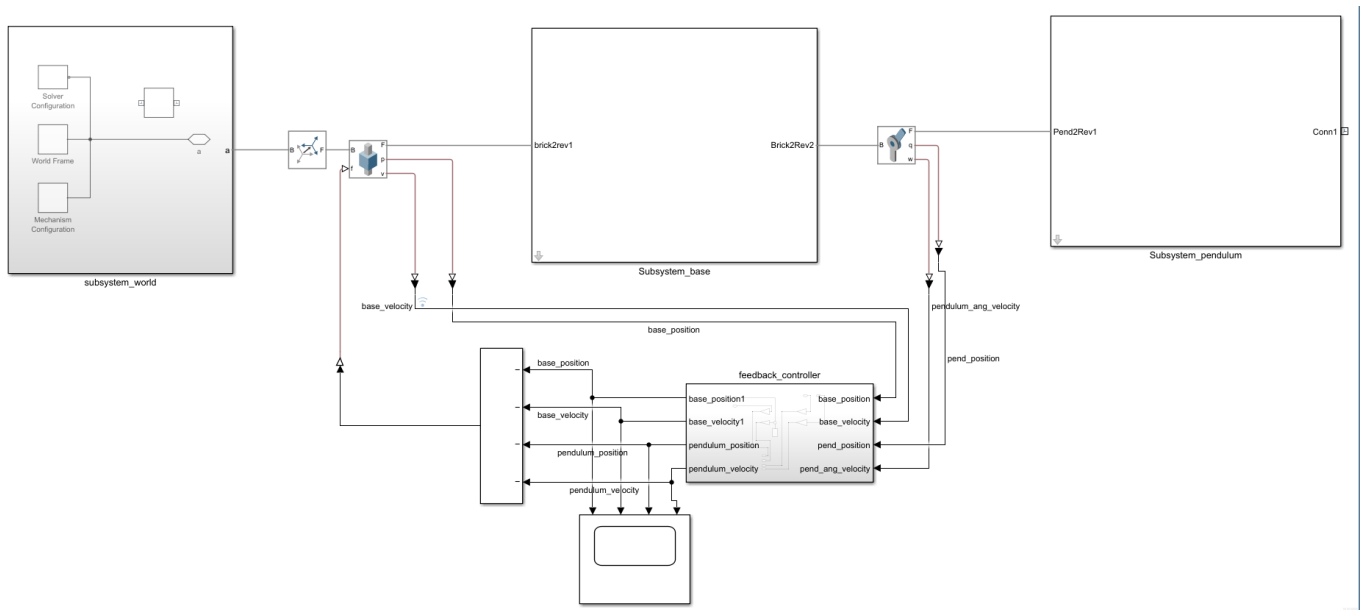


Figure 4-1. Final_simscape layout with feedback_controller

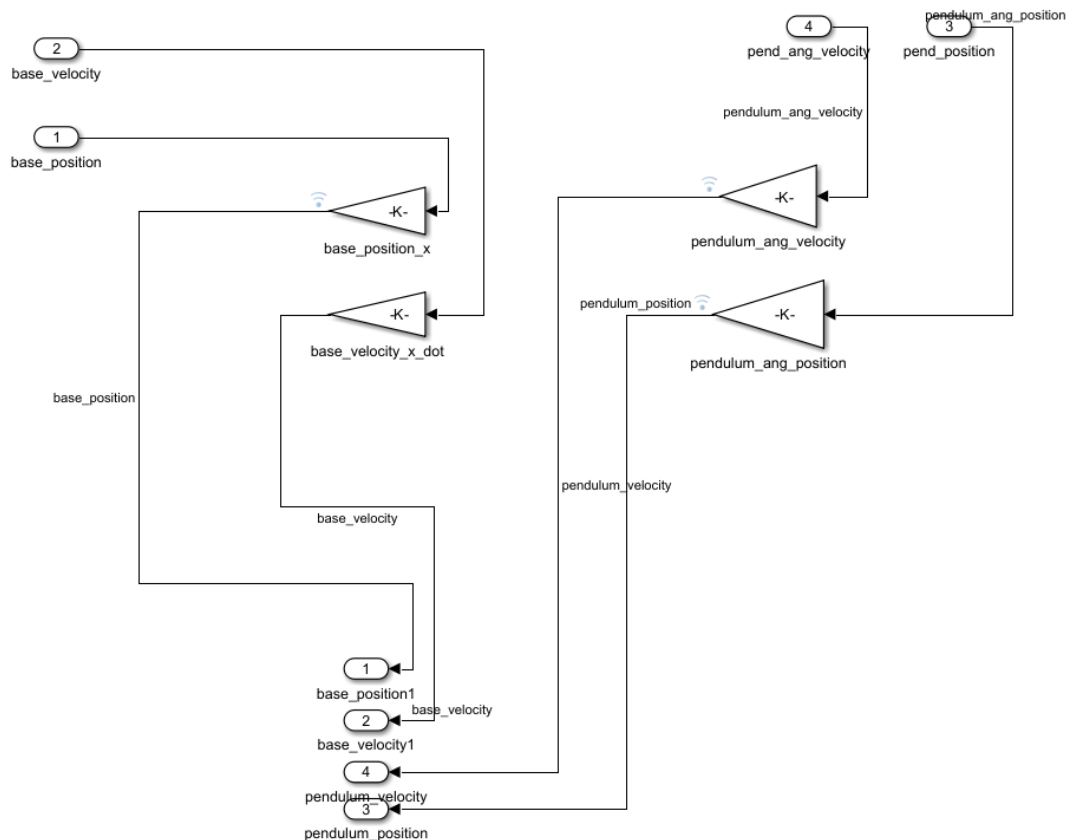


Figure 4-2. Basic feedback control structure for the inverted pendulum with four state variables and corresponding gain values

To begin with, all four gains were set to zero. I then started tuning them manually — a process often called trial-and-error tuning. The tuning followed this sequence:

1. Pendulum Angle Gain (K_3):

The first gain adjusted was for the pendulum angle. Starting with a small value (1 or -1), I increased the gain gradually until the pendulum began to move toward the upright position instead of falling over. This gain determines how strongly the controller reacts to the pendulum's tilt.

2. Pendulum Angular Velocity Gain (K_4):

Next, I introduced a gain for the pendulum's angular velocity. This term acts like damping — it helps smooth out the motion and prevents the pendulum from oscillating excessively.

3. Base Velocity Gain (K_2):

After stabilizing the pendulum's swing, I tuned the gain for the base velocity. This gain keeps the base from drifting too far horizontally. It

ensures that the base returns to its central position after correcting for the pendulum's tilt.

4. **Base Position Gain (K_1):**

Finally, I added a small gain for the base position to fine-tune the system's overall balance and prevent slow positional drift over time.

4.2.1 **Manual Gain Tuning and Results**

Before moving on to analytical tuning methods like LQR, I first tuned the feedback gains manually to get a better feel for how each state variable affects the overall system stability. This approach also helped verify that the model and control loop were functioning correctly.

After several iterations of testing and observing the response in the Simulink Data Inspector, I arrived at the following gain values:

Where:

- $K_1 = -0.7 \rightarrow$ Base position gain
- $K_2 = -2.1 \rightarrow$ Base velocity gain
- $K_3 = 25.5 \rightarrow$ Pendulum angle gain
- $K_4 = 4 \rightarrow$ Pendulum angular velocity gain

4.2.1.1 **Data Inspector Visualization and Analysis**

To evaluate the system's behaviour after applying the hand-tuned controller, I used the Simulink Data Inspector to visualize the simulation results. The Data Inspector made it easy to monitor how the pendulum angle, base position, and velocity changed over time.

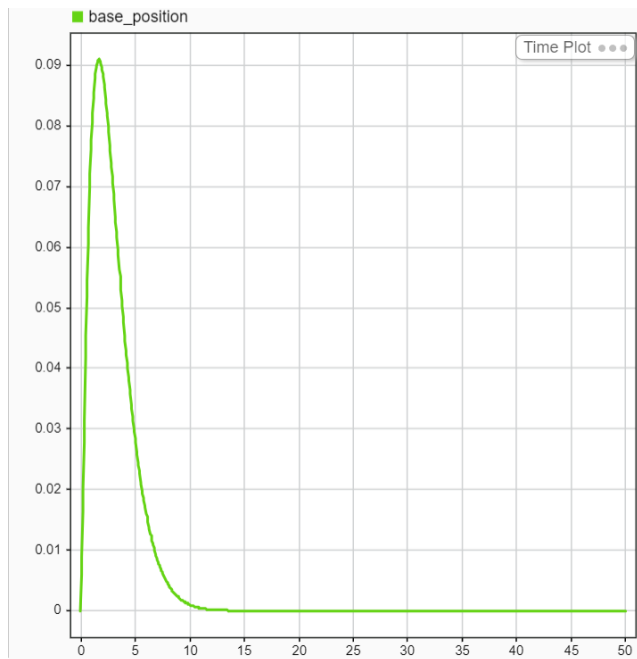


Figure 4-4 Base position response showing small corrective displacement and smooth return to equilibrium under hand-tuned control

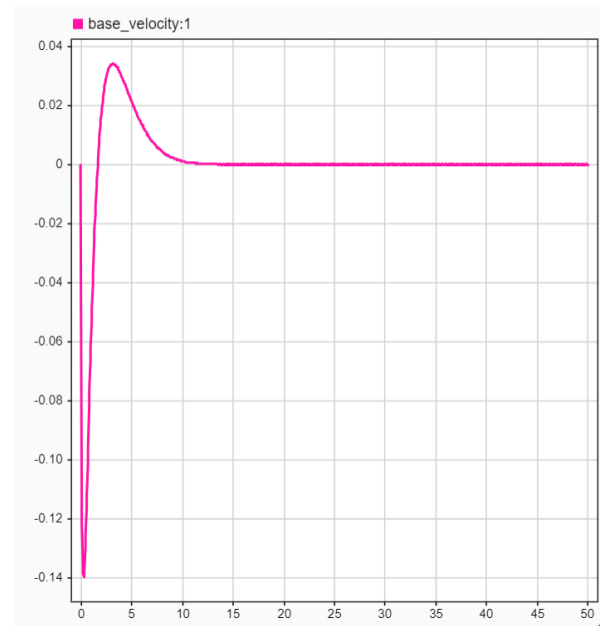


Figure 4-3 Base velocity response showing quick damping and steady-state stabilization after pendulum balancing

4.2.1.1.1 Base position

In the base position plot Figure 4-3, the base initially shifts slightly to counteract the pendulum's 5° offset. The peak displacement of about 0.09 meters occurs around 3–4 seconds before it gradually returns to zero. This behaviour indicates that the controller moves the base just enough to bring the pendulum upright and then holds it in position. The smooth return to zero without oscillation demonstrates good stability and minimal overshoot

4.2.1.1.2 base velocity plot

The base velocity plot Figure 4-3 further confirms this behaviour. Initially, there's a sharp increase in velocity as the base reacts to the falling pendulum. This is followed by a small overshoot, and then the velocity quickly settles near zero after approximately 10 seconds

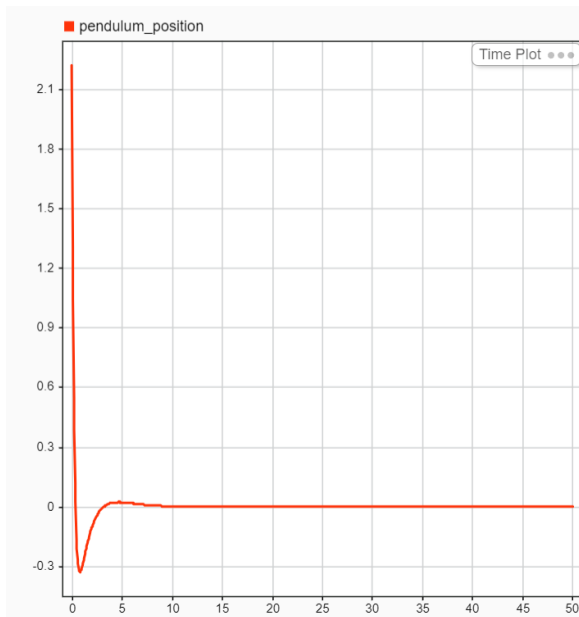


Figure 4-6: pendulum angular position

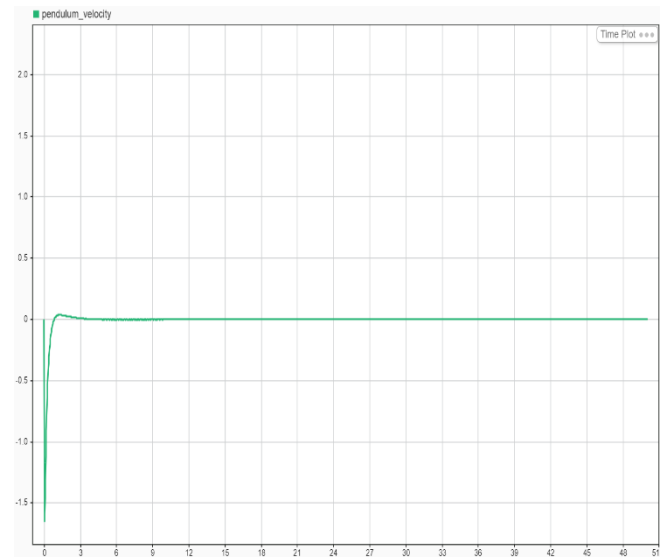


Figure 4-5: pendulum angular velocity

4.2.1.1.3 Pendulum angular position

In the pendulum angle plot (Figure 4-5), the pendulum initially starts at a 5° offset, causing a sharp change in angle at the beginning of the simulation. Immediately after that, the controller applies corrective action to bring it back toward the upright position. Within the first 5 seconds, the angle settles smoothly around zero with minimal overshoot, confirming that the controller effectively stabilizes the pendulum in the vertical position.

4.2.1.1.4 Pendulum Angular Velocity

The pendulum angular velocity plot (Figure 4-6) further supports this observation. The velocity shows an initial spike as the controller reacts to the tilt, followed by a rapid decrease as damping takes effect. After about 8–10 seconds, the angular velocity settles near zero and remains stable, meaning that the pendulum's motion has been completely balanced and there are no sustained oscillations.

4.3 Introducing Target Velocity to the Controlled Inverted Pendulum

After stabilizing the pendulum in the upright position, the next step was to make the system move intentionally while keeping its balance. In a real self-balancing robot, this would mean driving the wheels forward or backward while ensuring the robot doesn't tip over. To replicate this behaviour in the simulation, a target base velocity was introduced.

The concept is straightforward: instead of keeping the base stationary, the controller now tries to follow a desired velocity. This is achieved by feeding back the difference between the actual base velocity and the target velocity. The difference acts as an error signal, and the controller adjusts the base force to minimize it, causing the base to move while maintaining the pendulum's upright position.

To generate the target velocity, I used a Pulse Generator block in Simulink. The parameters were set as follows:

- Amplitude: 0.2
- Period: 4 seconds
- Pulse Width: 50%

This configuration produces an alternating forward and backward motion, where the base accelerates in one direction for 2 seconds and then reverses for the next 2 seconds. The signal was connected to the system through a

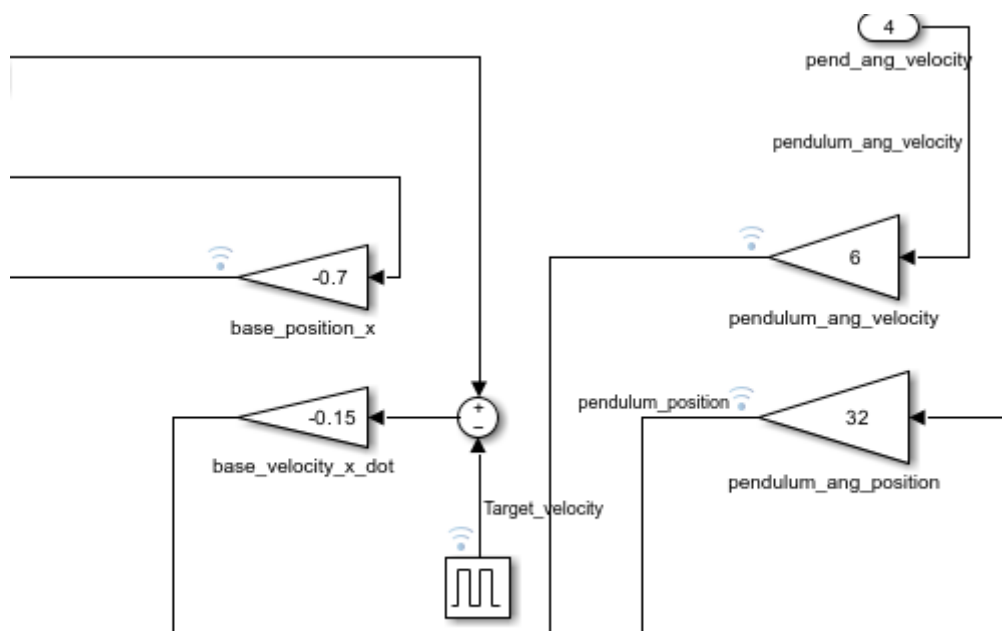


Figure 4-7 Implementation of target velocity using a Pulse Generator connected to the control loop

sum block, which computes the difference between the target velocity and the measured base velocity.

4.3.1 Controller Behavior with Hand-Tuned Gains and Target Velocity

Initially, I used the hand-tuned gains from the previous section to test the system's response when a target velocity was added. The gain values used were:

$$K = [-0.8, -0.5, 80, 2]$$

With these gains, the pendulum was able to maintain its upright position even when the base started to move according to the pulse input from the target velocity. The controller continuously adjusted the base movement to keep the pendulum balanced, demonstrating that the control logic was functioning as intended.

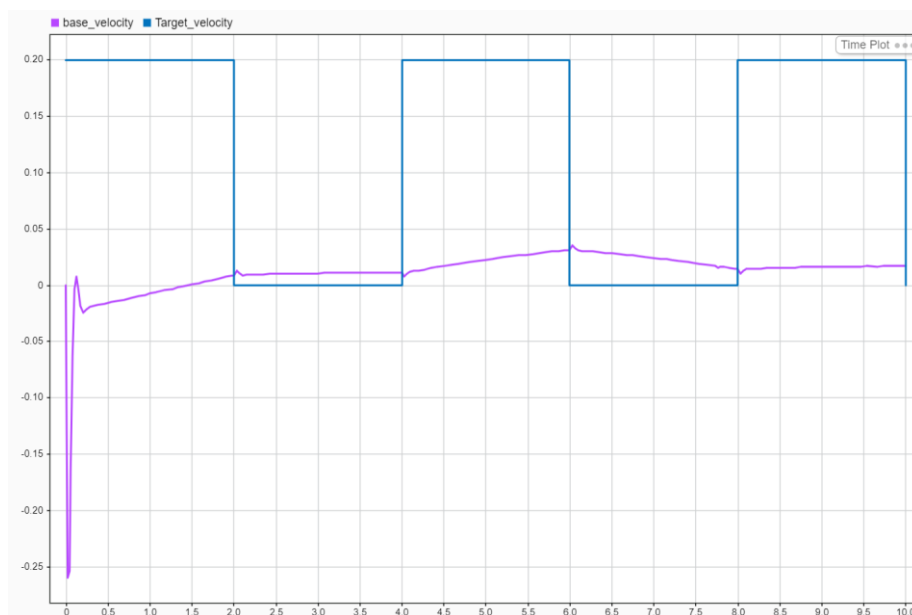


Figure 4-8 Comparison between base velocity and target velocity

The graph above illustrates the relationship between the target velocity and the actual base velocity after implementing the refined controller. The target velocity (blue) is generated using a Pulse Generator with an amplitude of 0.2 m/s, a period of 4 seconds, and a 50% duty cycle. The base velocity (magenta) follows this target input with a small delay. When the target velocity switches direction, the base responds immediately but with a

gradual change this prevents sudden jerks and keeps the pendulum stable throughout the motion.

Although the base velocity does not perfectly match the step-like pulse shape of the target velocity, it consistently follows the trend with steady, controlled acceleration and deceleration. This behaviour demonstrates that the controller not only maintains the pendulum's upright position but also ensures smooth and stable motion tracking during direction changes.

4.3.2 Base and Pendulum Position under Target Velocity

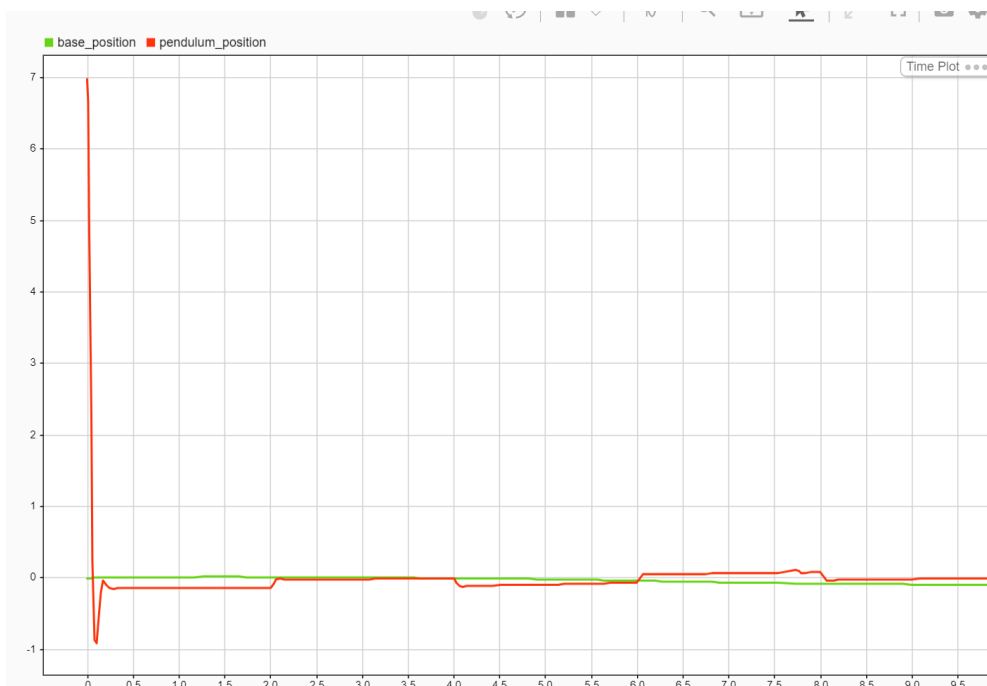


Figure 4-9 Comparison of base and pendulum positions showing stable motion

The graph above compares the base position (green) and pendulum position (red) over time after adding the target velocity input. At the start of the simulation, the pendulum begins with a small offset, causing a sharp change in its position as the controller immediately compensates to bring it back toward the upright state. Once the controller stabilizes the pendulum, both signals settle close to zero, indicating that the system maintains balance even while the base moves back and forth following the target velocity profile. Small position variations are visible at each pulse transition these correspond to the moments when the direction of the target velocity changes. Despite these shifts, the pendulum remains upright and stable, while the base adjusts its position smoothly without large deviations.

4.4 Linearizing the Simscape Multibody Model

To make the controller more accurate and responsive, I linearized the nonlinear Simscape Multibody model around its upright equilibrium point. Linearization basically simplifies the system's complex nonlinear motion into a linear form that's much easier to analyse and control mathematically.

I used the Model Linearizer tool in Simulink to perform this process at the initial operating point, where the pendulum is almost upright(5 degree offset). The resulting linearized model provided a state-space representation with one input, four outputs, and four states, describing the key system variables:

- x_1 : Base position
- x_2 : Base velocity
- x_3 : Pendulum angle
- x_4 : Pendulum angular velocity

After performing the linearization at the 5° off-balance position, the system was represented in its state-space form using the standard matrices A, B, C, and D.

Figure 4-10.MATRIX_A

$$A = \begin{matrix} & \begin{matrix} x1 & x2 & x3 & x4 \end{matrix} \\ \begin{matrix} x1 \\ x2 \\ x3 \\ x4 \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 8.675 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 145.1 & 0 \end{bmatrix} \end{matrix}$$

Figure 4-11.MATRIX_B

$$B = \begin{matrix} & u1 \\ \begin{matrix} x1 \\ x2 \\ x3 \\ x4 \end{matrix} & \begin{bmatrix} 0 \\ 3.129 \\ 0 \\ 24.47 \end{bmatrix} \end{matrix}$$

$$C = \begin{matrix} & \begin{matrix} x1 & x2 & x3 & x4 \end{matrix} \\ \begin{matrix} y1 \\ y2 \\ y3 \\ y4 \end{matrix} & \begin{bmatrix} -0.5 & 0 & 0 & 0 \\ 0 & -0.1 & 0 & 0 \\ 0 & 0 & 6 & 0 \\ 0 & 0 & 0 & 32 \end{bmatrix} \end{matrix}$$

$$D = \begin{matrix} & u1 \\ \begin{matrix} y1 \\ y2 \\ y3 \\ y4 \end{matrix} & \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \end{matrix}$$

Figure 4-13.MATRIX_C

Figure 4-12.MATRIX_D

4.4.1 Calculating the Feedback Gains in MATLAB

Once the system was linearized and the state-space matrices A, B, C, and D were obtained, I used MATLAB's LQR (Linear Quadratic Regulator) function to calculate the optimal feedback gains.

4.4.1.1 Code Used To Find Gains

```
% Define weighting matrices
```

```
Q = diag([10, 1, 100, 1]); % Emphasizes pendulum angle and base position stability
```

```
R = 1; % Penalizes excessive control effort
```

```
% Calculate LQR gain matrix
```

```
K = lqr(A, B, Q, R);
```

```
% Display result
```

```
disp('LQR Gain Matrix K = ');
```

```
disp(K);
```

- Q defines how strongly each state (position, velocity, angle, angular velocity) is weighted in the optimization.
- R defines the cost associated with applying control input (force).
- The `lqr()` function then returns the gain matrix K, which provides the optimal balance between stability, smooth response, and control effort.

The resulting gain values from this command were:

$$K = [-3.16, -3.22, 24.86, 2.14]$$

4.4.2 Base Velocity Response Using Model Linearizer Gains

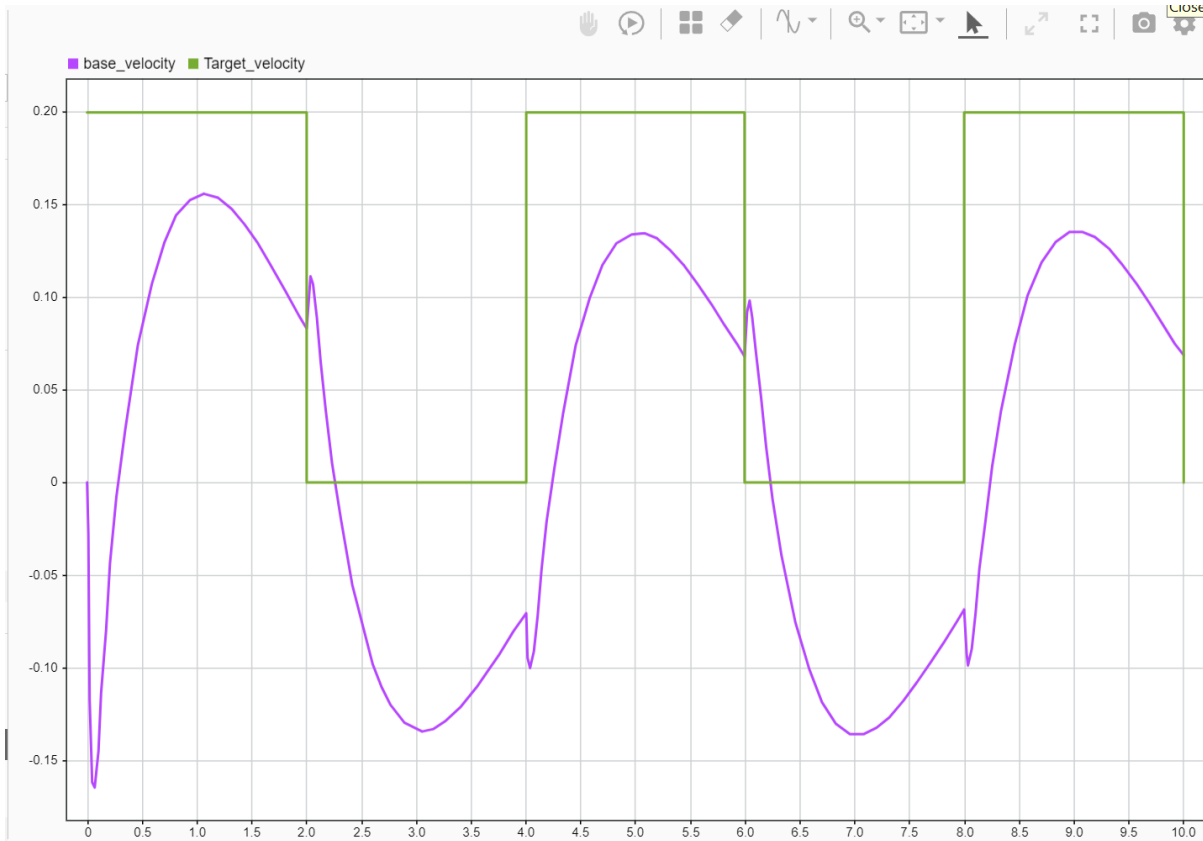


Figure 4-14 Comparison of base velocity (purple) and target velocity (green) using optimized gains obtained from Model Linearizer

The plot above shows the base velocity (purple) and target velocity (green) after implementing the optimized gain values obtained from the Model Linearizer. Compared to the hand-tuned results, the response is noticeably smoother and more stable. The base velocity follows the target signal more closely, with reduced oscillations and a faster settling behaviour after each direction change.

4.5 Optimizing the Control Parameters

After obtaining the optimized gains from the Model Linearizer, the next step was to further refine and analyse the controller's behaviour using MATLAB's Control System Designer App. This tool allows interactive tuning of controller parameters and provides immediate visual feedback on how changes in the gains affect system stability and response

4.6 Step Response Validation in Control System Tuner

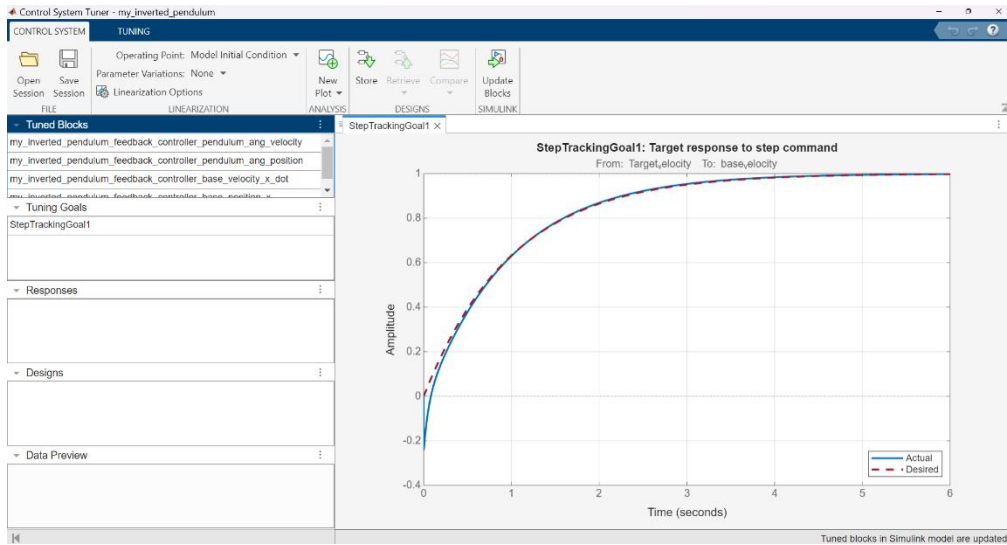


Figure 4-15. Step tracking response between target velocity and base velocity after tuning using Control System Tuner

The plot above shows the step tracking response between the target velocity (input) and the base velocity (output) obtained from the Control System Tuner. The blue line represents the actual system response, while the red dashed line indicates the desired reference trajectory. From the plot, it can be observed that the tuned controller achieves excellent tracking performance. The actual response closely follows the desired trajectory with minimal rise time and negligible overshoot. The smooth and steady convergence toward the final value confirms that the system is well-damped and stable.

4.6.1 System Response after Control System Tuning

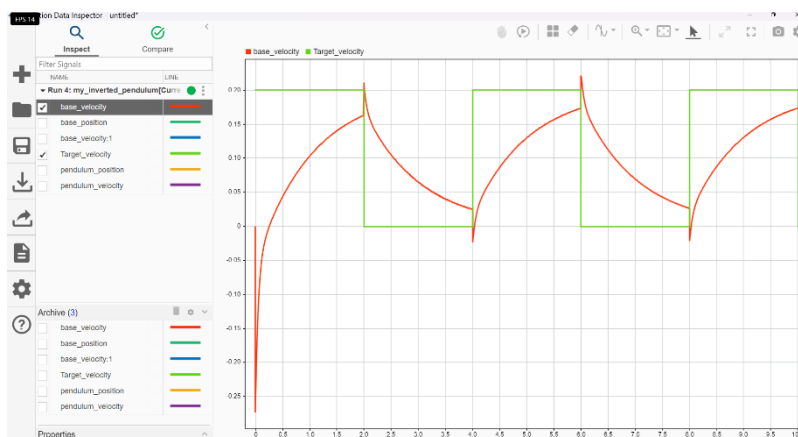


Figure 4-16. comparison of base velocity and target velocity after control system tuning

The graph above shows the base velocity (red) compared with the target velocity (green) after applying the optimized gains obtained from the Control System Tuner. The system demonstrates a much smoother and more accurate tracking response compared to earlier tuning methods. At each change in the target velocity (positive to negative and vice versa), the base velocity responds quickly and settles smoothly without overshoot or oscillation. The exponential rise and decay curves indicate well-damped behaviour, meaning the controller effectively manages both acceleration and deceleration phases.

The tuned gains have improved both stability and responsiveness — the pendulum remains balanced throughout, while the base movement stays consistent and closely follows the command profile. This confirms that the Control System Tuner provided the most refined performance, achieving a stable and realistic motion similar to what would be expected in a physical self-balancing system.